

Тезаурус по основным элементам курса компьютерной графики

1. Алгоритмические основы

- **Растреризация:**
 - Процесс преобразования векторной графики в растровую, что позволяет отображать изображения на экране. Включает определение, какие пиксели должны быть закрашены для представления графических примитивов.
- **Алгоритм Брезенхема:**
 - Оптимизированный алгоритм для рисования линий и окружностей, который использует целые числа для минимизации вычислений. Позволяет избежать плавающей запятой, что улучшает производительность.
- **Заливка (Flood Fill):**
 - Алгоритм, используемый для заполнения области цветом, начиная с заданной точки. Применяется в графических редакторах для изменения цвета областей.
- **Фрактальные алгоритмы:**
 - Генерация сложных форм и структур с использованием простых правил. Примеры включают фракталы Мандельброта и снежинку Коха, которые демонстрируют самоподобие.
- **Сглаживание:**
 - Методы, направленные на уменьшение "зубчатости" на границах объектов. Включает технологии, такие как Multisample Anti-Aliasing (MSAA) и Supersample Anti-Aliasing (SSAA).
- **Освещение:**
 - Модели, используемые для расчета взаимодействия света с поверхностями. Основные модели включают:
 - **Phong:** Модель, учитывающая диффузное и зеркальное отражение.
 - **Gouraud:** Модель, использующая интерполяцию цветов по вершинам многоугольников.
- **Трассировка лучей:**
 - Метод рендеринга, который обеспечивает реалистичное освещение и тени, моделируя путь света от источника до камеры. Используется для создания фотореалистичных изображений.

2. Графические примитивы

- **Точки и линии:**
 - Основные графические примитивы, используемые для построения форм. Точки представляют собой отдельные координаты, а линии — соединение двух точек.
- **Многоугольники:**
 - Замкнутые фигуры, состоящие из трех или более вершин. Многоугольники являются основными элементами для построения 2D и 3D объектов.

- **Кривые:**
 - Математические представления для создания плавных форм. Примеры включают кривые Безье и Нурбс, которые широко используются в компьютерной графике для моделирования.

3. Трансформации

- **Параллельные трансформации:**
 - Изменения положения объектов в пространстве, включая:
 - **Перемещение:** Изменение положения объекта по осям.
 - **Поворот:** Изменение ориентации объекта вокруг заданной оси.
 - **Масштабирование:** Изменение размера объекта по осям.
- **Отражения:**
 - Создание зеркальных изображений объектов, что позволяет визуализировать симметрию.

4. 3D-графика

- **Модели (Mesh):**
 - Структуры, представляющие 3D объекты. Состоят из вершин, рёбер и граней, которые формируют поверхность.
- **Сцены:**
 - Комплексы объектов, освещения и камер, которые создают визуальное представление 3D пространства.
- **Камеры:**
 - Определяют проекцию сцены на экран. Основные типы:
 - **Перспективная камера:** Создает эффект глубины, уменьшая размеры объектов с расстоянием.
 - **Ортогографическая камера:** Объекты сохраняют свои размеры независимо от расстояния.
- **Освещение:**
 - Различные типы освещения, влияющие на визуальный эффект:
 - **Окружающее освещение:** Обеспечивает базовый уровень света.
 - **Диффузное освещение:** Создает мягкие тени и освещает поверхность.
 - **Зеркальное освещение:** Создает блики и отражения на поверхности.

5. Создание графических моделей с помощью OpenGL

- **OpenGL:**
 - **Контекст OpenGL:** Среда для выполнения операций, необходимая для рендеринга.

- **Шейдеры:** Программы, написанные на языке GLSL, для обработки вершин и пикселей. Разделяются на:
 - **Вершинные шейдеры:** Обрабатывают вершины.
 - **Фрагментные шейдеры:** Обрабатывают пиксели.
- **Буферы (VBO, EBO):**
 - **VBO (Vertex Buffer Object):** Хранит данные о вершинах.
 - **EBO (Element Buffer Object):** Хранит индексы для оптимизации рендеринга.
- **Текстуры:** Изображения, наложенные на поверхности объектов, для создания реалистичных деталей.
- **Инструменты:**
 - **GLFW:** Библиотека для работы с окнами и вводом, упрощающая использование OpenGL.
 - **GLEW:** Обеспечивает доступ к расширениям OpenGL.
 - **GLM:** Математическая библиотека для работы с векторами и матрицами, необходимая для 3D графики.
- **Технологии:**
 - **Текстурирование:** Наложение текстур на 3D объекты для достижения реалистичности.
 - **Постобработка:** Улучшение визуального качества изображений после рендеринга, включая эффекты размытия, светофильтры и т.д.
 - **Картирование нормалей:** Техника для создания детализированных поверхностей, позволяющая имитировать мелкие детали без увеличения количества полигонов.

6. Программирование на C++ в среде Qt Creator

- **C++:**
 - **Синтаксис:** Основы написания кода, включая структуры, функции и классы.
 - **Объектно-ориентированное программирование:** Использование объектов для инкапсуляции данных и методов, позволяющее создавать более организованный и модульный код.
 - **Шаблоны:** Создание обобщенных функций и классов для переиспользования кода.
- **Qt:**
 - **Qt Widgets:** Библиотека для создания графических интерфейсов с использованием виджетов (кнопки, поля ввода и т.д.).
 - **Qt Quick (QML):** Декларативный язык для создания современных интерфейсов, позволяющий легко разрабатывать анимации и взаимодействия.
 - **Сигналы и слоты:** Механизм событий для взаимодействия объектов, позволяющий реализовать асинхронное программирование.

- **Работа с графикой:** Использование классов для рисования и управления 2D графикой, включая создание графических сцен и элементов.
- **Инструменты:**
 - **Qt Creator:** IDE для разработки приложений на Qt, включающая средства для написания, отладки и тестирования кода.

7. Программирование графических процессоров с использованием Direct3D и HLSL

- **Direct3D:**
 - **Введение в DirectX:** API для работы с графикой на Windows, обеспечивающий доступ к аппаратным возможностям графических карт.
 - **Создание устройства:** Инициализация графического устройства, необходимого для рендеринга сцен.
 - **Рендеринг:** Процесс отображения сцены на экране, включая настройку буферов и шейдеров.
- **HLSL (High-Level Shader Language):**
 - **Шейдеры:** Программы для обработки вершин и пикселей, определяющие визуальные характеристики объектов.
 - **Компиляция шейдеров:** Преобразование HLSL кода в исполняемый для GPU, что позволяет оптимизировать производительность.
- **Технологии:**
 - **Освещение:** Методы, такие как PBR (Physically Based Rendering), для реалистичного освещения объектов.
 - **Постобработка:** Техники для улучшения изображения после рендеринга, включая эффекты размытия, контрастности и цветокоррекции.
 - **Тени:** Создание реалистичных теней с помощью карт теней, что позволяет улучшить визуальную достоверность сцен.